

Specifica del Progetto: Programmazione Object Oriented e Basi di Dati

Anno Accademico 2025-26

Storico revisioni

Data	Versione	Autore/i	Descrizione
05/06/2026	0.2	Bernardo Breve, Riccardo Caccavale	Modificate le linee guida per la consegna del terzo homework (Sezione 8.1) e fornite istruzioni per le consegne svincolate (Sezione 7.1.1)
21/04/2026	0.1	Bernardo Breve, Riccardo Caccavale e Andrea Calì	Prima stesura.

1 Introduzione

Questo documento definisce la specifica estesa per un progetto congiunto di Basi di Dati e Programmazione Object Oriented da svolgere dagli studenti del primo anno del corso di laurea in Informatica. Il progetto sarà svolto in gruppi di due o tre studenti. Ogni gruppo dovrà ideare, progettare e implementare un piccolo sistema informativo su un tema a scelta, sviluppando:

- un database in PostgreSQL
- un'applicazione Java basata sul pattern BCE (Boundary-Control-Entity) + DAO (Data Access Object):
 - **Boundary** (gestione dell'interazione con la GUI)
 - **Control** (implementazione della logica applicativa e coordinamento dei componenti)
 - **Entity** (modelli di dominio e rappresentazione dei dati)
 - **DAO** (gestione della persistenza e dell'accesso ai dati)

2 Panoramica del progetto

Scopo: realizzare un sistema informativo completo, esercitando tecniche di analisi dei requisiti, modellazione dati, progettazione della base di dati, progettazione OO e implementazione.

Risultato richiesto: database PostgreSQL funzionante e applicazione Java a tre livelli.

3 Scelta del tema

Selezionate un dominio reale a vostra scelta, sufficientemente complesso da includere:

- Uno schema concettuale con almeno 5 entità/classi per gruppi di due studenti, o 7 per gruppi di tre.
- Almeno una generalizzazione

- Regole di business (vincoli semantici) non banali, che richiedano l'implementazione di almeno tre trigger.

Esempî:

- Gestione consegne di pacchi
- Prenotazione eventi
- Inventario e ordini
- Iscrizione e frequenza a corsi universitari

4 Obiettivi di apprendimento

- Raccogliere e documentare requisiti funzionali ed eventualmente non funzionali.
- Modellare casi d'uso e dominio di interesse con opportuni diagrammi UML/ER.
- Definire operazioni ad alto livello e loro mappatura a metodi Java, eventualmente su dati persistenti.
- Progettare schema concettuale (ER/UML) e tradurlo in schema relazionale con vincoli
- Implementare regole di integrità tramite vincoli di base, trigger o logica applicativa.
- Applicare principi OOP (incapsulamento, ereditarietà etc.) in Java.
- Produrre pseudo-codice chiaro per operazioni critiche.
- Scrivere codice funzionante nonché script SQL di creazione e popolamento dati.

5 Elaborati da consegnare

Per le attività di progetto sono richiesti, per ciascun insegnamento, gli elaborati dettagliati di seguito.

5.1 Programmazione Object Oriented

- Documento PDF contenente una descrizione di Progettazione O-O del sistema, composto da:
 - Diagramma delle classi UML del dominio del problema.
- Codice sorgente dell'applicativo
- File `pom.xml`, essendo richiesto l'utilizzo **obbligatorio** di Apache Maven.

5.2 Basi di Dati

1. Requisiti in linguaggio naturale
2. Diagramma dei casi d'uso con descrizioni testuali a supplemento
3. Lista delle funzioni ad alto livello
4. Schema concettuale (diagramma ER o UML)
5. Catalogo delle regole di business (vincoli semantici)
6. Schema concettuale ristrutturato
7. Schema logico del database con vincoli di base
8. Piano di implementazione dei vincoli sui dati (constraint, trigger, logica)
9. Script SQL per creazione dello schema (DDL), popolamento dati (DML) con dati di esempio, e definizione dei trigger.

Nota: Tutti gli elaborati da 1 a 8 devono essere inclusi in un unico PDF. Il codice Java e gli script SQL restano in file separati.

6 Formazione dei gruppi di lavoro

Sono ammessi gruppi di lavoro da 2 o 3 componenti. La registrazione del gruppo avviene tramite compilazione di un modulo come da istruzioni via Teams.

All'interno del modulo si richiede l'inserimento delle informazioni relative ai componenti del suddetto gruppo: nome, cognome, matricola ed un indirizzo di posta istituzionale (@studenti.unina.it). Per ogni gruppo registrato, il primo componente verrà considerato il referente del proprio gruppo; eventuali comunicazioni da parte dei docenti saranno inviate a detto referente, il quale dovrà farsi carico di informare i restanti componenti. Non sono previste modifiche ai gruppi di lavoro o progetti individuali a meno di giustificati motivi.

6.1 Comunicazioni e richieste al docente

Eventuali comunicazioni o richieste di chiarimenti ai docenti dovranno essere fatte separatamente per i due insegnamenti. I messaggi dovranno essere inviati dal referente del gruppo dall'indirizzo registrato e dovranno avere come destinatari in copia anche gli altri componenti del gruppo. È possibile effettuare comunicazioni tramite Teams; in tal caso creare una chat con il docente e gli altri membri del gruppo.

7 Istruzioni dettagliate

7.1 Programmazione Object Oriented

Strutturazione progetto: L'applicativo software da consegnare dovrà seguire il pattern di programmazione **BCE + DAO**. In particolare, è richiesto di garantire una chiara separazione tra la logica dell'interfaccia utente (GUI), la logica di business e il livello di persistenza dei dati. Per rendere esplicita tale separazione, si dovranno definire opportuni package:

- Package `gui` — contiene le classi appartenenti al *boundary*, responsabili dell'interazione con l'utente;
- Package `controller` — contiene le classi appartenenti al *control*, che implementano la logica applicativa e coordinano le operazioni;
- Package `model` — contiene le classi che definiscono le *entity*, ossia i modelli del dominio;
- Package `dao`, `dao.impl` e `db` — contengono rispettivamente le interfacce DAO, le loro implementazioni concrete e le classi per la gestione della connessione al database.

In particolare, gli studenti dovranno dimostrare la capacità di:

- Identificare correttamente le classi del dominio applicativo e le relative responsabilità.
- Organizzare il codice in modo ordinato e leggibile, usando nomi significativi per classi e metodi.
- Utilizzare il meccanismo di incapsulamento per proteggere lo stato interno degli oggetti, esponendo solo le operazioni necessarie tramite metodi pubblici.
- Riconoscere eventuali generalizzazioni, utilizzando ereditarietà e polimorfismo dove appropriato per modellare comportamenti comuni in maniera riusabile ed estendibile.
- Separare nettamente le responsabilità tra:
 - classi che rappresentano le strutture dati e le regole del dominio;
 - classi che orchestrano le interazioni tra interfaccia e logica di business;
 - classi che forniscono l'interfaccia grafica utente;
 - interfacce e classi che seguano il design pattern DAO, che incapsulano la logica di accesso ai dati.
- Gestire in modo modulare e flessibile la comunicazione tra i componenti per facilitare la manutenibilità del codice e l'eventuale sostituzione di componenti.

In sede di documentazione, oltre al diagramma UML delle classi e ai diagrammi di sequenza richiesti, si suggerisce (opzionale ma fortemente consigliato):

- una breve descrizione testuale delle principali classi, con focus su attributi, metodi e relazioni significative;
- un commento sulle scelte progettuali fatte in termini di riuso del codice, estendibilità, o separazione delle responsabilità.

Gestione delle eccezioni: Un'applicazione robusta deve essere in grado di gestire situazioni anomale o inattese senza interrompere il funzionamento complessivo del sistema. La progettazione deve quindi prevedere una corretta gestione delle eccezioni, in particolare:

- intercettare errori a livello di accesso al database (es. connessioni non riuscite, vincoli violati);
- validare gli input ricevuti dall'utente e segnalare eventuali valori non accettabili;
- propagare solo le eccezioni rilevanti ai livelli superiori, convertendole dove opportuno in messaggi chiari per l'utente finale;
- evitare che dettagli tecnici (per es. `SQLException`, `NullPointerException`) emergano nella GUI, mascherandole dietro messaggi controllati e comprensibili per l'utente.

Può essere inoltre utile definire classi di eccezione personalizzate per i casi d'uso rilevanti, come ad esempio `UtenteNonAutenticatoException`, al fine di rendere il flusso del codice più leggibile e manutenibile.

Utilizzo di librerie esterne: Durante lo sviluppo del progetto, è consentito l'utilizzo di librerie esterne non trattate durante il corso, purché compatibili con le specifiche del progetto. L'unico vincolo

obbligatorio è l'inclusione, nel file `pom.xml`, della dipendenza relativa al driver JDBC per PostgreSQL.

Utilizzo di strumenti di supporto al design dell'interfaccia: È ammesso l'impiego di strumenti software dedicati alla progettazione e realizzazione di interfacce grafiche (ad esempio, WindowBuilder). Tuttavia, l'utilizzo di tali strumenti **non esonera** i membri del gruppo dalla comprensione approfondita dei principi teorici e delle tecniche sottostanti alla progettazione e implementazione della GUI.

7.1.1 Istruzioni per la consegna separata del progetto di POO

Gli studenti che hanno dichiarato di voler svolgere separatamente i progetti di POO e di Basi di Dati possono presentare il progetto di POO anche in assenza di una base dati SQL sottostante.

Tuttavia, per evitare rallentamenti durante la presentazione del progetto, l'applicativo dovrà comunque contenere dati significativi con cui sia possibile interagire. A tal fine, gli studenti potranno gestire la persistenza dei dati mediante lettura e scrittura su file, utilizzando un formato a scelta tra quelli più comuni, ad esempio `.txt`, `.json`, `.csv` o altri formati equivalenti.

In questo caso, la valutazione terrà conto della capacità degli studenti di organizzare le informazioni in modo strutturato all'interno dei file e di leggerle correttamente quando necessario.

7.2 Basi di Dati

Specificazione dei requisiti

- Scopo e ambito del sistema
- Attori primari
- Requisiti funzionali (elenco casi d'uso)
- Eventuali requisiti non funzionali

Lista delle funzioni ad alto livello Raggruppate per modulo (es. Gestione Utenti, Gestione Ordini):

- Nome funzione
- Descrizione breve
- Input/Output principali

Schema concettuale Diagramma ER o UML con entità/classi, attributi, relazioni, cardinalità e gerarchie.

Regole di business Tutti i vincoli che devono sussistere sui dati, inclusi quelli semantici che non possono essere rappresentati con vincoli di base; per esempio:

- “Un ordine deve riferirsi a un cliente esistente.”
- “Se il cliente ha tessera *oro*, il prezzo finale dell'acquisto deve essere scontato del 30% rispetto a quello calcolato dal carrello; se ha tessera *argento*, lo sconto sarà del 15%”.
- “La giacenza di un prodotto non può scendere sotto zero.”

Schema concettuale ristrutturato Schema opportunamente ristrutturato per essere tradotto direttamente in schema relazionale.

Schema logico del database Tabelle con colonne, tipi, PK, FK, NOT NULL, UNIQUE, CHECK.

Diagramma dettagliato delle classi dell'applicazione Diagramma UML con package/pacchetti presentation, service, dao; classi, metodi (con segnatura), relazioni.

Specifiche dettagliate delle operazioni Per i casi d'uso più critici:

- Pseudo-codice
- Segnatura dei metodi
- Logica di business
- Operazioni su DB

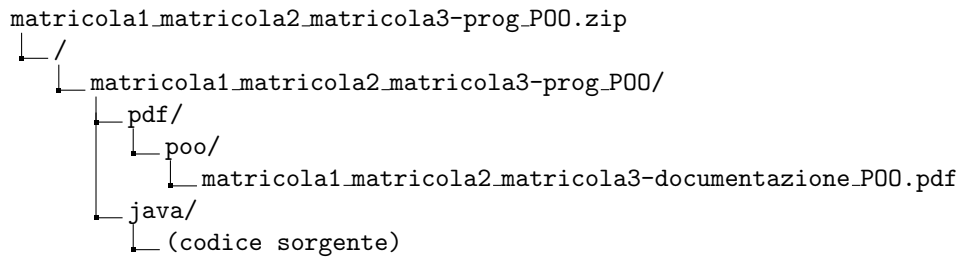


Figure 1: Struttura del file zip da consegnare. Si omette la terza matricola per i gruppi di due studenti.

Implementazione dei vincoli sui dati Per ogni regola di business (vincolo semantico), indicare se implementata tramite:

- Constraint SQL (PK, CHECK, FK)
- Trigger (specificandone eventi, condizione e azione)
- Validazione in applicazione, specificando classe/metodo e suo collocamento nell'architettura.

8 Modalità di consegna

Le modalità di consegna previste per i due insegnamenti sono *del tutto indipendenti*. Tuttavia, la struttura dei file di progetto è la medesima ed è descritta in Figura 1.

8.1 Programmazione Object Oriented

La consegna dell'elaborato di progetto avviene tramite Teams. Di seguito sono descritte le modalità di consegna, distinte in base allo svolgimento del progetto secondo la modalità *in itinere* oppure *completa*.

Consegna secondo la modalità in itinere In corrispondenza della scadenza di ciascun homework, i docenti provvederanno a creare un'attività dedicata sul canale Teams del corso. Tramite tale attività, il referente del progetto dovrà caricare gli elaborati richiesti per ciascun homework, secondo le seguenti indicazioni:

- *Primo homework*: un file PDF contenente una descrizione del sistema, una presentazione del dominio scelto per lo svolgimento del progetto, il link alla repository GitHub del progetto e il diagramma delle classi relativo al dominio del problema.
- *Secondo homework*: la release GitHub corrispondente allo stato del codice e della documentazione alla scadenza prevista per l'homework. In questa fase, il codice dovrà già essere organizzato secondo il pattern BCE+DAO e dovrà includere una bozza dell'interfaccia grafica, correttamente collegata al controller.
- *Terzo homework*: la release GitHub corrispondente allo stato finale del codice e della documentazione alla scadenza prevista per l'homework. In questa fase, il progetto dovrà risultare completo in ogni sua parte, sia dal punto di vista del codice sia della documentazione. Per i progetti svolti secondo la modalità *in itinere*, la consegna del terzo homework dovrà essere effettuata nelle attività relative ai singoli appelli d'esame. Affinché il progetto venga considerato svolto secondo la modalità *in itinere*, esso deve essere discusso dal gruppo **entro la discussione prevista per l'appello di settembre**.

Consegna secondo la modalità completa In corrispondenza di ciascun appello d'esame, i docenti provvederanno a creare un'attività dedicata sul canale Teams del corso, tramite la quale il referente del progetto potrà caricare la release GitHub.

Nota: Il progetto dovrà essere caricato obbligatoriamente *almeno tre giorni prima* della data fissata per la discussione.

9 Modalità di presentazione e valutazione

9.1 Istruzioni in comune tra i due insegnamenti

- Il documento non deve essere una mera sequenza degli elaborati elencati nella sezione 5, ma una relazione sul progetto nella sua interezza.
- Si ricorda che i due insegnamenti sono indipendenti. Non c'è quindi alcun vincolo o propedeuticità tra i due.
- La discussione consisterà in una dimostrazione del sistema realizzato, della durata di *al più 15 minuti*, nonché da domande dei docenti a tutti i membri del gruppo. Si sottolinea infine che, in sede di discussione del progetto, ***tutti i membri del gruppo devono avere cognizione di ogni parte del progetto.***

9.2 Istruzioni per l'insegnamento di Programmazione Object Oriented

- Il progetto discusso e presentato dovrà essere scaricato dagli studenti direttamente dall'attività corrispondente su Teams.
- Sarà valutata la qualità della progettazione delle classi, anche in termini di responsabilità e ruoli di ciascuna di esse nell'intero progetto.
- Sarà oggetto di valutazione la corretta separazione della parte di logica applicativa dalla logica di definizione, manipolazione e interrogazione dei dati.
- Sarà altresì oggetto di valutazione anche la qualità dell'esperienza utente nell'utilizzo dell'applicativo software